

Project deployment

1. Mục tiêu

Tài liệu trình bày lại quá trình đưa dự án Laravel lar_vidu2 từ máy cá nhân lên Internet, sử dụng Render để chạy ứng dụng và Aiven để lưu dữ liệu MySQL. Nội dung dựa trên cấu hình dự án và những vấn đề đã xử lý trong quá trình triển khai.

Mô hình thực hiện: phát triển và kiểm tra trên máy cá nhân; đưa mã nguồn lên GitHub; Render lấy mã nguồn và tạo môi trường Docker; ứng dụng kết nối đến MySQL trên Aiven.

2. Công cụ và thuật ngữ cần biết

Công cụ / thuật ngữ	Định nghĩa và vai trò trong dự án
Composer	Công cụ quản lý thư viện PHP. Tập tin composer.lock giúp cài đúng các phiên bản thư viện của dự án.
Git và GitHub	Git quản lý các phiên bản thay đổi; GitHub lưu kho mã nguồn trực tuyến để Render lấy về triển khai.
Docker	Công cụ đóng gói ứng dụng cùng môi trường chạy. Image là bản đóng gói; container là môi trường đang chạy từ image đó.
Dockerfile và entrypoint	Dockerfile mô tả cách tạo image. Entrypoint là tập tin thực hiện các bước chuẩn bị và khởi động container.
Nginx và PHP-FPM	Nginx nhận yêu cầu từ trình duyệt và phục vụ nội dung web; PHP-FPM xử lý mã PHP của Laravel.
Render	Dịch vụ chạy ứng dụng trên Internet. Dự án sử dụng Web Service với môi trường Docker.
Aiven	Dịch vụ cơ sở dữ liệu được quản lý. Trong dự án, Aiven cung cấp MySQL để lưu tài khoản, sản phẩm và đơn hàng.

Công cụ / thuật ngữ	Định nghĩa và vai trò trong dự án
Biến môi trường	Các giá trị cấu hình riêng cho từng nơi chạy, như địa chỉ website, thông tin MySQL và khóa ứng dụng.
Secret File	Tập tin được khai báo riêng trên Render và cung cấp cho ứng dụng khi chạy. Dự án dùng cơ chế này để đưa chứng chỉ CA vào container.
SSL/TLS và chứng chỉ CA	SSL/TLS bảo vệ kết nối bằng mã hóa. Chứng chỉ CA giúp ứng dụng xác minh chứng chỉ máy chủ MySQL của Aiven.
Migration và seeder	Migration quản lý cấu trúc bảng. Seeder tạo dữ liệu ban đầu, như tài khoản admin, danh mục và sản phẩm mẫu.
Proxy, log và health check	Proxy đứng giữa người truy cập và ứng dụng; log ghi lại hoạt động và lỗi; health check là yêu cầu kiểm tra ứng dụng có phản hồi hay không.

3. Trình tự triển khai

Bước 1. Kiểm tra dự án trên máy

1. Mở đúng thư mục lar_vidu2 trong trình soạn thảo.
2. Kiểm tra PHP đáp ứng yêu cầu dự án và các thư viện Composer đã được cài đầy đủ.
3. Kiểm tra tập tin môi trường local, tên cơ sở dữ liệu và thông tin đăng nhập MySQL trên máy.
4. Bật MySQL trong XAMPP; kiểm tra cơ sở dữ liệu đã có các bảng cần thiết.
5. Khởi động máy chủ phát triển Laravel; mở trang chủ, sản phẩm và đăng nhập để kiểm tra.
6. Sau khi thay đổi cấu hình local, xóa cấu hình Laravel đã lưu đệm và khởi động lại máy chủ phát triển.

Lưu ý: cấu hình MySQL local và Aiven phải được quản lý riêng. Với MySQL local thông thường, để trống MYSQL_ATTR_SSL_CA. Không xóa cấu hình CA trên Render để xử lý lỗi ở máy cá nhân.

Lệnh thực hiện tại máy — PowerShell, trong thư mục dự án:

Set-Location C:\xampp\htdocs\lar_vidu2
--

```
php --version
composer install
php artisan config:clear
php artisan migrate:status
```

Lệnh Composer cài thư viện theo composer.lock. Lệnh cuối chỉ xem tình trạng migrations; nếu cơ sở dữ liệu local còn migration chưa chạy và bạn muốn áp dụng chúng. Thì chạy lại php artisan migrate để cập nhật hết các trang mirage

Trích cấu hình .env dành cho MySQL local: sửa các khóa tương ứng, giữ các cấu hình khác của dự án.

```
APP_ENV=local
APP_DEBUG=true
APP_URL=http://localhost:8000
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=TEN_DATABASE_LOCAL
DB_USERNAME=root
DB_PASSWORD=
MYSQL_ATTR_SSL_CA=
SESSION_SECURE_COOKIE=false
```

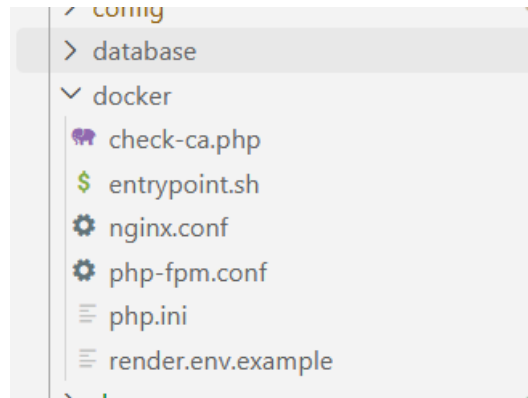
TEN_DATABASE_LOCAL là giá trị cần thay. Tài khoản root và mật khẩu trống chỉ minh họa cấu hình XAMPP thường gặp; dùng thông tin thật của MySQL trên máy. Không áp dụng APP_DEBUG hoặc cookie HTTP local cho Render.

Bước 2. Chuẩn bị các tập tin triển khai

1. Kiểm tra Dockerfile để xác định phiên bản PHP, các extension, thư viện và thành phần máy chủ cần dùng.
2. Kiểm tra cấu hình Nginx phục vụ từ thư mục public và sử dụng cổng do môi trường triển khai cung cấp.
3. Kiểm tra cấu hình PHP và PHP-FPM, đồng thời bảo đảm ứng dụng ghi được vào storage và bootstrap/cache.
4. Chuẩn bị entrypoint để kiểm tra cấu hình, xử lý chứng chỉ, chạy migrations, chạy seeders khi được bật và khởi động máy chủ.
5. Kiểm tra .dockerignore để loại cấu hình riêng, thư viện local, log và tập tin không cần thiết khỏi quá trình build.
6. Kiểm tra .gitignore để tránh đưa tập tin môi trường và thông tin nhạy cảm lên GitHub.

Các tập tin chính: Dockerfile, .dockerignore, docker/nginx.conf, docker/php.ini, docker/php-fpm.conf, docker/entrypoint.sh, docker/check-ca.php và docker/render.env.example.

Lưu ý: .gitignore kiểm soát việc Git theo dõi tập tin; .dockerignore kiểm soát dữ liệu gửi vào build Docker. Hai tập tin không thay thế cho nhau.



Cấu trúc thư mục docker

a. Dockerfile — nội dung hiện tại: chia bước cài thư viện và môi trường chạy thành các giai đoạn riêng.

```
FROM php:8.2-fpm-alpine AS php-base

RUN apk add --no-cache bash nginx curl gettext su-exec tini ca-certificates \
    libpng libzip oniguruma \
    && apk add --no-cache --virtual .build-deps $PHPIZE_DEPS \
    libpng-dev libzip-dev oniguruma-dev \
    && docker-php-ext-install -j"$(nproc)" pdo_mysql mbstring zip gd bcmath opcache \
    && apk del .build-deps

WORKDIR /var/www

FROM php-base AS build
COPY --from=composer:2 /usr/bin/composer /usr/local/bin/composer
COPY composer.json composer.lock ./
RUN composer install --no-dev --prefer-dist --no-interaction --no-progress \
```

```

    --no-scripts --no-autoloader
COPY . .
RUN mkdir -p bootstrap/cache storage/framework/cache/data \
    storage/framework/sessions storage/framework/views storage/logs storage/app/public \
    && composer dump-autoload --no-dev --optimize --no-interaction \
    && composer check-platform-reqs --no-dev

FROM php-base AS production
ENV APP_ENV=production APP_DEBUG=false LOG_CHANNEL=stderr LOG_LEVEL=info \
    DB_CONNECTION=mysql SESSION_DRIVER=database SESSION_SECURE_COOKIE=true \
    CACHE_STORE=database QUEUE_CONNECTION=sync PORT=10000 RUN_MIGRATIONS=true

COPY --from=build --chown=www-data:www-data /var/www /var/www
COPY docker/nginx.conf /etc/nginx/templates/default.conf.template
COPY docker/php.ini /usr/local/etc/php/conf.d/zz-app.ini
COPY docker/php-fpm.conf /usr/local/etc/php-fpm.d/zz-app.conf
COPY --chmod=755 docker/entrypoint.sh /usr/local/bin/app-entrypoint
RUN mkdir -p /run/nginx \
    && chmod -R ug+rwX /var/www/storage /var/www/bootstrap/cache

EXPOSE 10000
HEALTHCHECK --interval=30s --timeout=5s --start-period=60s --retries=3 \
    CMD curl --fail --silent "http://127.0.0.1:${PORT}/up" > /dev/null || exit 1
ENTRYPOINT ["/sbin/tini", "--", "/usr/local/bin/app-entrypoint"]

```

Giai đoạn build dùng Composer cài thư viện cần cho production; image cuối nhận ứng dụng cùng vendor. PHP 8.2 và các extension phù hợp yêu cầu dự án. Tini quản lý tiến trình đầu của container; entrypoint chịu trách nhiệm chuẩn bị ứng dụng. Cấu hình môi trường thật được cung cấp khi chạy. Giao diện hiện chưa dùng Vite nên image chưa cần bước build bằng Node.

b. docker/nginx.conf — nội dung hiện tại: chỉ thực thi PHP thông qua điểm vào index.php của Laravel.

```

server {
    listen 0.0.0.0:${PORT};
    server_name _;

```

```

root /var/www/public;
index index.php;
charset utf-8;
server_tokens off;

access_log /dev/stdout;
error_log /dev/stderr warn;
client_max_body_size 10m;

location / {
    try_files $uri $uri/ /index.php?$query_string;
}

# Only the Laravel front controller may execute PHP.
location = /index.php {
    include fastcgi_params;
    fastcgi_param SCRIPT_FILENAME $document_root/index.php;
    fastcgi_param HTTP_PROXY "";
    fastcgi_pass 127.0.0.1:9000;
    fastcgi_hide_header X-Powered-By;
}

location ~ /\.php$ {
    return 404;
}

location ~ /\.(!well-known).* {
    deny all;
}
}

```

PORT được entriypoint điền vào cấu hình lúc khởi động. try_files chuyển các đường dẫn ứng dụng về Laravel; các quy tắc cuối chặn thực thi tập tin PHP khác và truy cập tập tin ẩn.

c. **docker/php.ini** và **docker/php-fpm.conf** — các thiết lập sử dụng: php.ini, khối thứ hai thuộc php-fpm.conf.

php.ini: là file cấu hình tổng thể của trình biên dịch/thông dịch PHP. Nó áp dụng cho cách mã nguồn PHP thực thi, bất kể chạy qua CLI, Apache hay PHP-FPM.

```
expose_php=Off
display_errors=Off
log_errors=On
error_log=/proc/self/fd/2
memory_limit=128M
upload_max_filesize=8M
post_max_size=10M
max_execution_time=60
opcache.enable=1
opcache.validate_timestamps=0
opcache.memory_consumption=64
opcache.max_accelerated_files=10000
```

php-fpm.conf. hoạt động như một dịch vụ máy chủ đứng sau Nginx. Nó nhận các yêu cầu FastCGI từ Nginx, phân phát cho các tiến trình con (worker) để xử lý PHP, rồi trả kết quả về Nginx.

```
[www]
listen = 127.0.0.1:9000
clear_env = no
catch_workers_output = yes
decorate_workers_output = no
pm = ondemand
pm.max_children = 3
pm.process_idle_timeout = 10s
pm.max_requests = 500
```

PHP ghi lỗi vào log thay vì hiện ra trình duyệt. `clear_env` cho phép tiến trình PHP-FPM nhận biến môi trường; số tiến trình con được giới hạn cho môi trường ít tài nguyên.

d. Trích `docker/entrypoint.sh` — làm mới cache và khởi tạo dữ liệu: đoạn này chạy sau khi chúng chỉ, cấu hình và quyền ghi đã được chuẩn bị.

```
#!/bin/bash
set -Eeuo pipefail

cd /var/www

# Render secret mounts may be readable by root but not by www-data.
# Copy only the CA certificate at runtime to an app-readable private location.
if [[ -n "${MYSQL_ATTR_SSL_CA:-}" ]]; then
    if [[ ! -f "$MYSQL_ATTR_SSL_CA" || ! -r "$MYSQL_ATTR_SSL_CA" ]]; then
        echo "Cannot read MySQL CA file. Check Render Secret Files and MYSQL_ATTR_SSL_CA." >&2
        exit 1
    fi
    (
        umask 077
        mkdir -p /run/app-certificates
        chown root:www-data /run/app-certificates
        chmod 750 /run/app-certificates
        cp "$MYSQL_ATTR_SSL_CA" /run/app-certificates/mysql-ca.pem
        chown www-data:www-data /run/app-certificates/mysql-ca.pem
        chmod 400 /run/app-certificates/mysql-ca.pem
    )
    export MYSQL_ATTR_SSL_CA=/run/app-certificates/mysql-ca.pem
    su-exec www-data php docker/check-ca.php
fi

# Allow maintenance commands with: docker run ... IMAGE php artisan ...
if (( $# > 0 )); then
    exec su-exec www-data "$@"
fi
```

```

: "${APP_KEY:?Set a persistent APP_KEY before starting the application}"
: "${APP_URL:?Set APP_URL to the public HTTPS address}"
export PORT="${PORT:-10000}"
if [[ ! "$PORT" =~ ^[0-9]{1,5}$ ]] || (( 10#$PORT < 1 || 10#$PORT > 65535 )); then
    echo "PORT must be an integer between 1 and 65535" >&2
    exit 1
fi
# Substitute PORT only; preserve Nginx variables such as $uri and $query_string.
envsubst '${PORT}' < /etc/nginx/templates/default.conf.template > /etc/nginx/http.d/default.conf

mkdir -p storage/framework/{cache/data,sessions,views} storage/logs storage/app/public bootstrap/cache
chown -R www-data:www-data storage bootstrap/cache

su-exec www-data php artisan config:cache
case "${RUN_MIGRATIONS:-true}" in
    true) su-exec www-data php artisan migrate --force --no-interaction ;;
    false) ;;
    *) echo "RUN_MIGRATIONS must be true or false" >&2; exit 1 ;;
esac
case "${RUN_SEEDERS:-false}" in
    true) su-exec www-data php artisan db:seed --force --no-interaction ;;
    false) ;;
    *) echo "RUN_SEEDERS must be true or false" >&2; exit 1 ;;
esac
su-exec www-data php artisan route:cache
su-exec www-data php artisan view:cache

nginx -t
php-fpm -t

# Stop the whole container if either server exits, and forward stop signals.
server_pids=()
cleanup() {
    trap - EXIT TERM INT

```

```

if (( ${#server_pids[@]} )); then
    kill -QUIT "${server_pids[@]}" 2>/dev/null || true
    wait "${server_pids[@]}" 2>/dev/null || true
fi
}
trap cleanup EXIT
trap 'exit 0' TERM INT

php-fpm -F &
server_pids+=("${!}")
nginx -g 'daemon off;' &
server_pids+=("${!}")

status=0
wait -n "${server_pids[@]}" || status=$?
echo "A web server exited (status $status); stopping container" >&2
exit 1

```

File script `docker/entrypoint.sh` đóng vai trò là điểm điều phối khởi động cốt lõi giúp container vận hành an toàn và tự động hóa toàn bộ quy trình triển khai trên nền tảng Render. Trước tiên, script thiết lập chế độ kiểm soát lỗi nghiêm ngặt (`set -Eeuo pipefail`), đồng thời trích xuất và phân quyền chặt chẽ cho chứng chỉ SSL kết nối tới cơ sở dữ liệu Aiven MySQL. Tiếp đó, hệ thống tự động kiểm tra các biến môi trường bắt buộc, nạp giá trị cổng `$PORT` động vào file cấu hình Nginx và cấp quyền ghi cho thư mục storage của ứng dụng. Sau khi môi trường sẵn sàng, script tối ưu hóa hiệu năng Laravel bằng cách đóng băng cache cấu hình, route, giao diện, đồng thời tự động thực thi các bản di chuyển dữ liệu (migrate) vào database. Trước khi tiếp nhận lưu lượng truy cập, cú pháp của cả Nginx và PHP-FPM đều được kiểm tra kỹ lưỡng để loại trừ lỗi cấu hình. Cuối cùng, hai dịch vụ này được kích hoạt song song và giám sát chặt chẽ thông qua cơ chế bắt tín hiệu (trap), đảm bảo nếu một trong hai tiến trình gặp sự cố thì container sẽ lập tức dừng lại để hệ thống tự động phục hồi.

e. Trích `.dockerignore` — các mục loại khỏi build: giữ cả những quy tắc khác đã có trong tập tin.

```

.git
.github
.agents

```

```
.codex
.idea
.vscode
.env
.env.*
**/.env
**/.env.*
auth.json
vendor
node_modules
tests
.phpunit.cache
.phpunit.result.cache
storage
bootstrap/cache
public/hot
public/storage
database/*.sqlite*
**/*.sql
**/*.pem
**/*.key
**/*.log
test.cs
docker/*.env*
```

Không chuyển nguyên các quy tắc này sang .gitignore: tập tin môi trường mẫu vẫn cần được quản lý bằng Git. Nếu muốn kiểm tra image tại máy đã cài Docker, có thể build thử; đây là bước tùy chọn, chưa phải kết quả kiểm tra được ghi nhận trong lần chuẩn bị trước.

`docker build -t lar-vidu2 .`

Bước 3. Đưa mã nguồn lên GitHub

1. Kiểm tra kho GitHub, địa chỉ remote và nhánh sẽ dùng để triển khai.
2. Xem danh sách thay đổi; chọn đủ mã nguồn và các tập tin triển khai liên quan.

3. Kiểm tra không có mật khẩu, khóa dịch vụ hoặc nội dung tập tin môi trường thật trong những thay đổi sẽ đưa lên.
4. Tạo một phiên bản lưu, gọi là commit, với mô tả rõ nội dung cập nhật.
5. Đẩy commit lên GitHub, gọi là push.
6. Mở GitHub và xác nhận nhánh triển khai đã có đúng phiên bản mới nhất.

Kết quả cần đạt: Render có thể lấy đầy đủ dự án từ GitHub. Những tập tin chỉ tồn tại trên máy và chưa được đưa lên Git sẽ không xuất hiện trong bản triển khai.

Lệnh Git tại máy — áp dụng cho kho hiện đã có remote:

```
git status --short
git branch --show-current
git remote -v
git diff --stat
```

Sau khi xem nội dung thay đổi và xác nhận không có bí mật, đưa các nhóm tập tin triển khai vào phiên bản lưu:

```
git status --short
git branch --show-current
git remote -v
git diff --stat
```

Hai lệnh add chọn những nhóm tập tin phục vụ đợt triển khai này. Nếu có sửa controller hoặc view, chọn thêm đúng tập tin đó. Lệnh push minh họa nhánh main; đối chiếu với nhánh đang làm việc và nhánh Render sử dụng. Không tạo lại origin nếu kho đã được kết nối, không dùng force push cho quy trình cập nhật thông thường.

Bước 4. Chuẩn bị cơ sở dữ liệu Aiven (aiven.io)

1. Đăng nhập Aiven và tạo dịch vụ MySQL theo gói đã chọn cho dự án.

Home Projects Tools Billing Support Admin My Organization

Create service

Project: phamhonghai-3257 Organization: My Organization

Select service type

**PostgreSQL®**
PostgreSQL - High-performance relational database with advanced extensions

**Apache Kafka®**
Kafka - Distributed event streaming platform for high-throughput data pipelines

**OpenSearch®**
OpenSearch - Distributed real-time search and analytics

**ClickHouse®**
ClickHouse - Fast resource-effective data warehouse for analytical workloads

**Valkey**
Valkey - High-performance key/value datastore

**Dragonfly**
Dragonfly - Scalable in-memory data store for high-performance workloads

**Aiven for Metrics**
Thanos Metrics - Scalable Prometheus query solution

**MySQL**
MySQL - Popular general-purpose easy-to-use relational database

**Grafana®**
Grafana - Data visualization and analytics platform

Apache, Apache Kafka, Kafka, Apache Flink, and Flink are either registered trademarks or trademarks of the Apache Software Foundation in the United States and/or other countries. ClickHouse, OpenSearch, PostgreSQL, MySQL, Grafana, Dragonfly, Valkey, Terraform, and Kubernetes are trademarks and property of their respective owners. All product and service names used in this website are for identification purposes only and do not imply endorsement.

Cancel

- nhấn creat MySQL

Create service

Project: phamhonghai-3257 Organization: My Organization

Cloud

You can select a specific cloud provider and region on the Professional tier.

Asia Pacific

Australia

Europe

North America

Plan

Plan	VMs	CPUs per VM	RAM per VM	Storage	Monthly price
Free-1-1gb	1	1	1 GB	1 GB	Free

Service basics

Name*

The service name cannot be changed afterwards

mysql-1a302a19

Add tag to this service

Service summary

Service

MySQL

Name

mysql-1a302a19

Service tier

Free

Cloud

Asia Pacific

Plan

Free-1-1gb

1 CPU

1 GB RAM

1 GB storage

Backups for disaster recovery

Monthly price

Free

Back

Create service

- Chọn cloud gần khu vực Việt Nam
 - Đặt tên Service
 - Chọn bản Free
- Chờ dịch vụ sẵn sàng, sau đó mở thông tin kết nối của đúng dịch vụ.
 - Ghi nhận host, cổng, tên cơ sở dữ liệu, tên người dùng và mật khẩu.

Home Projects Tools Billing Support Admin My Organization

Aiven platform trial is active. 27 days left to use \$50.00 USD trial credits on non-free plan services. [Upgrade options](#)

PHAMHONGHAI-3257

- Overview
- Connect
 - Databases
 - Integrations
 - Users
- Observe
 - Metrics
 - Logs
 - AI insights
 - Query statistics
 - Current queries
- Backups
- Service settings

Connection information

[Quick connect](#)

MySQL	MySQLx
Service URI	mysql://CLICK_TO_REVEAL_PASSWORD@mysql-3b72-phamhonghai-3257.i.aivencloud.com:16933/defaultdb?ssl-mode=REQUIRED
Database name	defaultdb
Host	mysql-3b72-phamhonghai-3257.i.aivencloud.com
Port	16933
User	avnadmin
Password	*****
SSL mode	REQUIRED
CA certificate	Show

Service plan usage [Upgrade plan](#)

4. Tải chứng chỉ CA do dịch vụ cung cấp; giữ nguyên nội dung chứng chỉ.

The screenshot shows the Aiven console interface. At the top, a dark green banner states: "Aiven platform trial is active. 27 days left to use \$50.00 USD trial credits on non-free plan services. [Upgrade options](#)". The navigation bar includes "Home", "Projects" (selected), "Tools", "Billing", "Support", and "Admin". The user's organization is "My Organization".

The left sidebar contains the following menu items: Overview, Connect, Databases, Integrations, Users, Observe, Metrics, Logs, AI insights, Query statistics, Current queries, Backups, and Service settings.

The main content area is titled "Connection information" and has a "Quick connect" button. It displays details for a MySQL service:

	MySQL	MySQLx
Service URI	mysql://CLICK_TO_REVEAL_PASSWORD@mysql-3b72ff21-phamhonghai-3257.i.aivencloud.com:16933/defaultdb?ssl-mode=REQUIRED	
Database name	defaultdb	
Host	mysql-3b72ff21-phamhonghai-3257.i.aivencloud.com	
Port	16933	
User	avnadmin	
Password	*****	
SSL mode	REQUIRED	
CA certificate	Show	

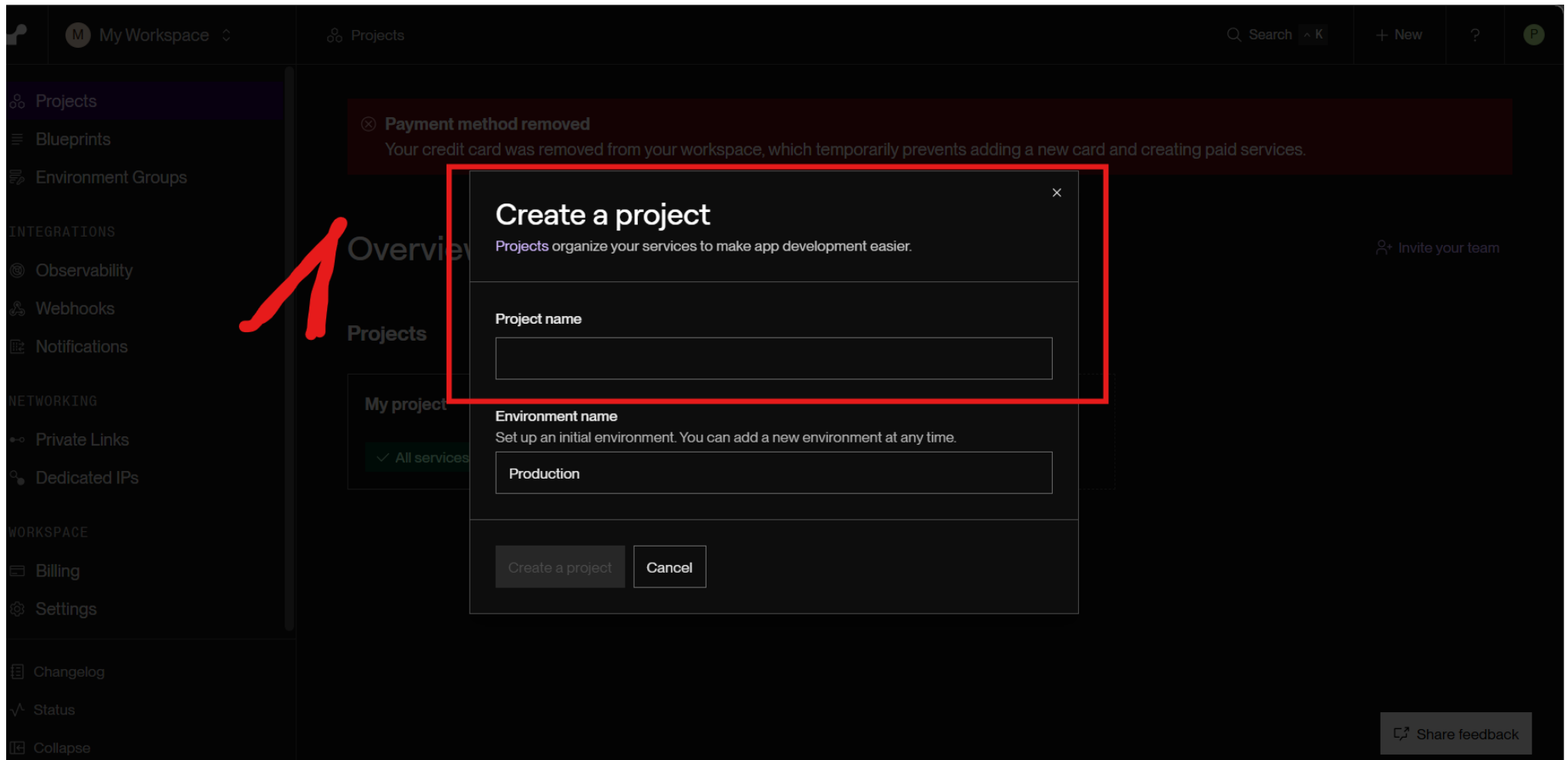
A red box highlights the download and copy icons at the bottom right of the CA certificate row.

5. Chuẩn bị sử dụng thông tin này ở phần cấu hình Render.

Lưu ý: không dùng localhost hoặc tự lấy cổng MySQL trên máy thay cho thông số Aiven. Đưa mã nguồn lên GitHub không đồng thời chuyển dữ liệu local lên Aiven. Trong quy trình này, migrations tạo bảng và seeders bổ sung dữ liệu mẫu; nếu muốn giữ dữ liệu local, cần thêm bước xuất và nhập dữ liệu riêng.

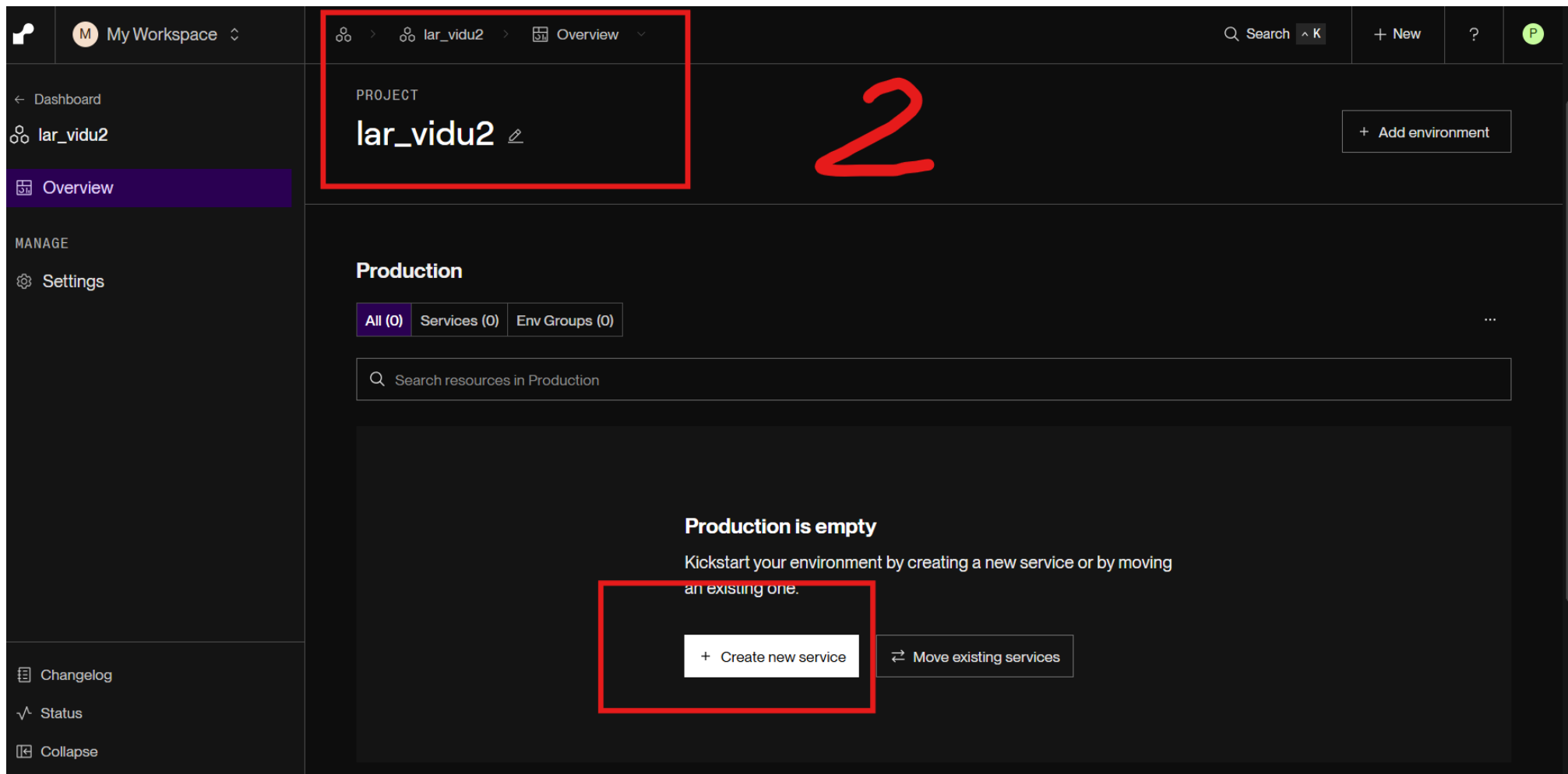
Bước 5. Tạo Web Service trên Render (<https://render.com>)

1. Đăng nhập Render và kết nối tài khoản GitHub.

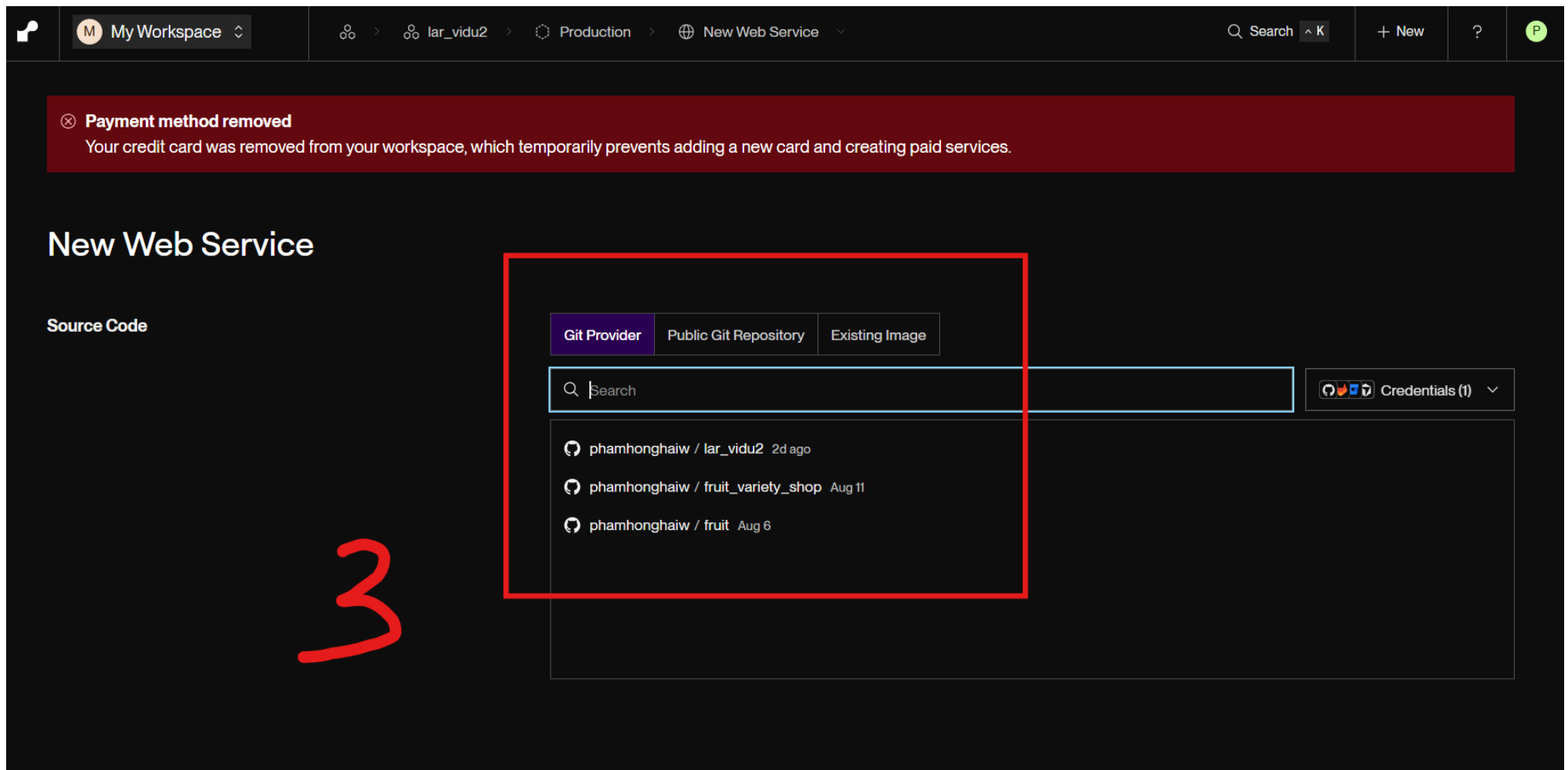


- Đặt tên dự án

2. Chọn tạo Web Service, chọn kho lar_vidu2 và đúng nhánh triển khai.



3. Kiểm tra Root Directory theo cấu trúc thật của kho mã nguồn. Nếu dự án nằm ngay tại gốc kho, không thêm một tầng thư mục không tồn tại.



4. Chỉ định Dockerfile ở gốc dự án và ngữ cảnh build tương ứng.

- Language: Docker
- Branch: main
- Region: Singapore

5. Giữ lệnh khởi động theo image đã chuẩn bị, không ghi đè bằng một lệnh khác.

6. Đặt đường dẫn health check là /up.
7. Ghi nhận địa chỉ HTTPS mà Render cấp để khai báo địa chỉ ứng dụng.

Lưu ý: sử dụng Web Service vì Laravel cần chạy PHP phía máy chủ. Đường dẫn /up kiểm tra khả năng phản hồi của Laravel, không kiểm tra toàn bộ chức năng bán hàng.

Bước 6. Khai báo biến môi trường và chứng chỉ

Mở mục Environment trên Render, đối chiếu tập tin docker/render.env.example và điền các nhóm thông tin sau:

Nhóm cấu hình	Thao tác cần thực hiện
Ứng dụng	Đặt môi trường production, tắt debug, khai báo APP_URL bằng địa chỉ HTTPS thật và thiết lập APP_KEY ổn định.
MySQL	Nhập các giá trị DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME và DB_PASSWORD lấy từ Aiven.
Kết nối SSL	Đặt MYSQL_ATTR_SSL_CA trỏ đến /etc/secrets/ca.pem.
Proxy và phiên đăng nhập	Khai báo TRUSTED_PROXIES phù hợp dịch vụ Render; lưu session bằng database và dùng cookie bảo mật trên HTTPS.
Cache và khởi tạo	Sử dụng cache database; bật RUN_MIGRATIONS. Chỉ bật RUN_SEEDERS khi cần tạo dữ liệu mẫu.
Tích hợp	Điền thông tin email, MoMo và GHN theo chức năng sử dụng và đúng môi trường thử nghiệm hoặc thật.

Tiếp theo, thêm chứng chỉ:

1. Mở phần Secret Files của Render và tạo tập tin tên ca.pem.
2. Dán đầy đủ chứng chỉ Aiven, gồm cả dòng bắt đầu và kết thúc.
3. Kiểm tra tên tập tin khớp với đường dẫn đã khai báo; không sử dụng đường dẫn Windows trên Render.
4. Lưu và áp dụng cấu hình bằng một lần triển khai mới.

Secret File được cung cấp trong thư mục /etc/secrets khi ứng dụng chạy. Entrypoint của dự án sao chép chứng chỉ đến vị trí có quyền đọc phù hợp cho tiến trình PHP và kiểm tra định dạng trước khi kết nối. [Tài liệu Secret File của Render](#)

Lưu ý: APP_KEY cần được giữ ổn định qua các lần triển khai. Trong ô giá trị của biến môi trường, chỉ điền giá trị tương ứng. Nếu chỉ lưu cấu hình mà chưa áp dụng bằng lần triển khai mới, ứng dụng đang chạy chưa nhận thay đổi. [Tài liệu biến môi trường Render](#)

Mẫu biến môi trường nhập trên Render: đây là ví dụ cần thay thông tin, không phải thông tin đăng nhập đang sử dụng. Phần seeder ban đầu để tắt; bật theo Bước 7.

```
APP_NAME=lar_vidu2
APP_ENV=production
APP_DEBUG=false
APP_URL=https://YOUR-SERVICE.onrender.com
APP_KEY=THAY_BANG_APP_KEY_DA_TAO
TRUSTED_PROXIES=*
DB_CONNECTION=mysql
DB_HOST=YOUR-AIVEN-HOST
DB_PORT=YOUR-AIVEN-PORT
DB_DATABASE=defaultdb
DB_USERNAME=avnadmin
DB_PASSWORD=THAY_BANG_MAT_KHAU_AIVEN
MYSQL_ATTR_SSL_CA=/etc/ssl/certs/ca.pem
SESSION_DRIVER=database
SESSION_SECURE_COOKIE=true
CACHE_STORE=database
QUEUE_CONNECTION=sync
LOG_CHANNEL=stderr
LOG_LEVEL=info
```

```
PORT=10000
RUN_MIGRATIONS=true
RUN_SEEDERS=false
MAIL_MAILER=log
MAIL_FROM_ADDRESS=hello@example.com
MAIL_FROM_NAME=lar_vidu2
```

Ví dụ:

Environment Variables

Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more.](#)

KEY	VALUE
APP_DEBUG	true
APP_ENV	production
APP_KEY	base64-j/k3oTqR3C[REDACTED]
APP_NAME	LaravelShop
APP_URL	https://laratestshop.onrender.com
DB_CONNECTION	mysql
DB_DATABASE	defaultdb
DB_HOST	mysql-3b72ff21-[REDACTED]
DB_PASSWORD	AVNS_2R1GS-CuKfk7Seuno1w
DB_PORT	18933
DB_USERNAME	avnadmin
MYSQL_ATTR_SSL_CA	/etc/ssl/certs/ca.pem
RUN_SEEDERS	*****
SEED_ADMIN_EMAIL	admin@example.com
SEED_ADMIN_NAME	*****

Bước 7. Khởi tạo bảng, dữ liệu mẫu và admin

1. trên Render.
2. Nhập email hợp lệ và mật khẩu riêng dài ít nhất 12 ký tự.
3. Bật RUN_SEEDERS cho lần khởi tạo dữ liệu.
4. Triển khai ứng dụng; quá trình khởi động chạy migrations trước, sau đó chạy seeders.
5. Đọc log để kiểm tra bước tạo bảng và tạo tài khoản admin.
6. Đăng nhập bằng email, mật khẩu đã khai báo; kiểm tra danh mục và sản phẩm mẫu.
7. Sau khi khởi tạo thành công, tắt RUN_SEEDERS cho các lần triển khai thông thường.

Dữ liệu được chuẩn bị: một admin khi chưa tồn tại, ba danh mục và mười sản phẩm mẫu. Admin này được đánh dấu đã xác thực để người quản trị có thể đăng nhập ngay. Tài khoản khách hàng đăng ký thông thường vẫn theo quy trình xác thực email.

Lưu ý: không có mật khẩu admin cố định trong mã nguồn. Seeder giữ nguyên tài khoản admin đã tồn tại; đổi biến mật khẩu không đặt lại mật khẩu trong cơ sở dữ liệu. Seeder cũng không tự nâng quyền một tài khoản thường trùng email. Việc khởi tạo được thực hiện qua entryptpoint vì gói Render Free không cung cấp Shell/SSH cho dịch vụ. [Giới hạn Render Free](#)

a. Cấu hình khởi tạo trên Render — thay email và mật khẩu mẫu trước khi dùng:

```
RUN_SEEDERS=true  
SEED_ADMIN_NAME="Shop Admin"  
SEED_ADMIN_EMAIL=admin@example.com  
SEED_ADMIN_PASSWORD=THAY_BANG_MAT_KHAU_RIENG
```

Địa chỉ admin@example.com chỉ minh họa định dạng; dùng email bạn quản lý. Khi nhập tên vào ô Value riêng của Render, nhập Shop Admin không kèm dấu nháy. Sau lần tạo thành công, đổi RUN_SEEDERS về false.

b. Tạo mới config/seeding.php — đọc thông tin admin từ môi trường:

```
<?php  
  
return [  
    'admin' => [  

```

```

        'name' => env('SEED_ADMIN_NAME', 'Shop Admin'),
        'email' => env('SEED_ADMIN_EMAIL'),
        'password' => env('SEED_ADMIN_PASSWORD'),
    ],
];

```

c. database/seeder/DatabaseSeeder.php — gọi seeders theo thứ tự:

```

<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\DB;

class DatabaseSeeder extends Seeder
{
    public function run(): void
    {
        DB::transaction(function () {
            $this->call([
                AdminUserSeeder::class,
                CategorySeeder::class,
                ProductSeeder::class,
            ]);
        });
    }
}

```

Transaction giúp các thao tác ghi dữ liệu trong lượt seed cùng thành công hoặc được hoàn tác khi có lỗi. Danh mục được tạo trước sản phẩm để lấy đúng mã danh mục.

d. Trích AdminUserSeeder.php — kiểm tra email và tài khoản đã tồn tại: nằm trong phương thức run; lớp sử dụng User, Hash và RuntimeException như tập tin hiện có.

```

<?php

namespace Database\Seeders;

use App\Models\User;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\Hash;
use RuntimeException;

class AdminUserSeeder extends Seeder
{
    public function run(): void
    {
        $email = trim((string) config('seeding.admin.email'));

        if (! filter_var($email, FILTER_VALIDATE_EMAIL)) {
            throw new RuntimeException('Set a valid SEED_ADMIN_EMAIL before running seeders.');
        }

        $existing = User::where('email', $email)->first();
        if ($existing) {
            if ($existing->role !== 'admin') {
                throw new RuntimeException('SEED_ADMIN_EMAIL belongs to a non-admin account. Choose a different
email.');
            }

            $this->command?->info('Admin already exists; existing account and password kept.');
            return;
        }

        $password = (string) config('seeding.admin.password');
        if (strlen($password) < 12) {
            throw new RuntimeException('Set SEED_ADMIN_PASSWORD to at least 12 characters before creating the
admin.');
        }
    }
}

```

```

    $admin = new User();
    $admin->forceFill([
        'name' => config('seeding.admin.name') ?: 'Shop Admin',
        'email' => $email,
        'password' => Hash::make($password),
        'role' => 'admin',
        // This account is provisioned by the operator, without sending email.
        'email_verified_at' => now(),
    ]->save();

    $this->command?->info('Admin created and verified. Sign in with the configured seed credentials.');
```

Phần tiếp theo trong cùng phương thức — tạo admin mới:

Hash::make băm mật khẩu trước khi lưu. Chỉ admin khởi tạo có email_verified_at được thiết lập ngay; không áp dụng cách này để bỏ xác thực cho khách hàng. Nếu email đã thuộc một admin, phương thức kết thúc trước bước tạo nên không đổi mật khẩu cũ.

e. Trích CategorySeeder.php và ProductSeeder.php — phần thân phương thức run: đoạn thứ nhất tạo danh mục; đoạn thứ hai tạo đủ mười sản phẩm mẫu.

```

class CategorySeeder extends Seeder
{
    public function run(): void
    {
        foreach (['Quần', 'Áo', 'Phụ kiện'] as $name) {
            Category::firstOrCreate(['name' => $name]);
        }
    }
}
```

```

class ProductSeeder extends Seeder
```

```

{
    public function run(): void
    {
        $products = [
            ['Quần jean nam', 'Quần', 350000, 100],
            ['Áo thun nữ', 'Áo', 250000, 80],
            ['Phụ kiện thời trang', 'Phụ kiện', 150000, 120],
            ['Áo sơ mi nam', 'Áo', 300000, 50],
            ['Quần short nữ', 'Quần', 280000, 60],
            ['Balo mini', 'Phụ kiện', 400000, 40],
            ['Áo khoác jean', 'Áo', 500000, 30],
            ['Vòng tay đá', 'Phụ kiện', 180000, 70],
            ['Quần tây nam', 'Quần', 320000, 90],
            ['Áo hoodie', 'Áo', 450000, 55],
        ];

        foreach ($products as [$name, $categoryName, $price, $quantity]) {
            $category = Category::firstOrCreate(['name' => $categoryName]);

            Product::firstOrCreate(
                ['name' => $name, 'category_id' => $category->id],
                ['price' => $price, 'quantity' => $quantity],
            );
        }
    }
}

```

firstOrCreate tìm bản ghi theo các thuộc tính nhận diện và chỉ thêm khi chưa có. Giá và tồn kho của sản phẩm đã tồn tại được giữ nguyên; không dùng seeder như chức năng cập nhật sản phẩm hằng ngày.

f. Trích migration tạo bảng sessions — phần bên trong phương thức up: dự án đã có tập tin 2026_09_21_000000_create_sessions_table.php, không tạo thêm migration trùng bảng.

```

public function up(): void

```

```
{
  Schema::create('sessions', function (Blueprint $table) {
    $table->string('id')->primary();
    $table->foreignId('user_id')->nullable()->index();
    $table->string('ip_address', 45)->nullable();
    $table->text('user_agent')->nullable();
    $table->longText('payload');
    $table->integer('last_activity')->index();
  });
}
```

Bảng này hỗ trợ SESSION_DRIVER dùng database. Để thử seed trên máy, khai báo SEED_ADMIN tương ứng trong môi trường local, kiểm tra đúng database, rồi thực hiện:

```
php artisan config:clear
php artisan db:seed
```

Trên Render, dùng cờ khởi động đã trình bày; không cần truy cập Shell để chạy các lệnh local này.

Bước 8. Theo dõi triển khai và kiểm tra kết quả

1. Mở log triển khai để theo dõi quá trình tạo image, cài thư viện và khởi động container.
2. Nếu có lỗi, đọc thông báo cụ thể trước phần danh sách gọi hàm; phân biệt lỗi build và lỗi khi ứng dụng bắt đầu chạy.

My Workspace

My project > Production > lartestshop > Deploys

Payment method removed
Your credit card was removed from your workspace, which temporarily prevents adding a new card and creating paid services.

WEB SERVICE
lartestshop Docker Free Upgrade your instance →

Service ID: `srv-dapknju7bico73frglog`

phamhonghaiw / lar_vidu2 main → f804ed9 Live

<https://lartestshop.onrender.com>

Auto-Deploy has been disabled to prevent accidental deploys. This service was rolled back to a previous deploy. [Auto-Deploy Settings](#)

Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more. [Upgrade now](#)

Search deploys and commits

DEPLOY #17	TRIGGER	DURATION
✓ Add deployment seeding for admin and sample products f804ed9 · Deployed 1d ago by you	Manual	1m02s
✗ Add deployment seeding for admin and sample products f804ed9 · Deployed 1d ago by you	Manual	36, 2s Rollback
✓ Fix Laravel startup and Render SSL certificate access efe367e · Deployed 1d ago by you	Rollback	20, 5s Rollback
✓ Add deployment seeding for admin and sample products f804ed9 · Deployed 1d ago	Auto-Deploy	43, 8s Rollback
✓ Fix Laravel startup and Render SSL certificate access efe367e · Deployed 1d ago by you	Manual	5m19s Rollback
✓ Fix Laravel startup and Render SSL certificate access efe367e · Deployed 1d ago	Auto-Deploy	1m18s Rollback
✗ Add a certificate and update db ssl config		

3. Khi dịch vụ sẵn sàng, mở URL công khai và kiểm tra trang chủ, sản phẩm, đăng nhập và trang admin.

Payment method removed
Your credit card was removed from your workspace, which temporarily prevents adding a new card and creating paid services.

WEB SERVICE
lartestshop Docker Free Upgrade your instance →

Service ID: `srv-dapknju7bko73trglog`

phamhonghaiw / lar_vidu2 main → f804ed9 ✓ Live
https://lartestshop.onrender.com

Auto-Deploy has been disabled to prevent accidental deploys. This service was rolled back to a previous deploy. [Auto-Deploy Settings](#)

Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more. [Upgrade now](#)

DEPLOY	TRIGGER	DURATION
✓ Add deployment seeding for admin and sample products f804ed9 - Deployed 1d ago by you	Manual	1m02s
✗ Add deployment seeding for admin and sample products f804ed9 - Deployed 1d ago by you	Manual	36, 2s Rollback
✓ Fix Laravel startup and Render SSL certificate access efe367c - Deployed 1d ago by you	Rollback	20, 5s Rollback
✓ Add deployment seeding for admin and sample products f804ed9 - Deployed 1d ago	Auto-Deploy	43, 8s Rollback
✓ Fix Laravel startup and Render SSL certificate access efe367c - Deployed 1d ago by you	Manual	5m19s Rollback
✓ Fix Laravel startup and Render SSL certificate access efe367c - Deployed 1d ago	Auto-Deploy	1m18s Rollback

4. Tạo một đơn hàng thử để kiểm tra giỏ hàng, đơn hàng và dữ liệu hiển thị trong Finance.
5. Kiểm tra riêng email, thanh toán và vận chuyển nếu các tích hợp đã được cấu hình.
6. Với MoMo và GHN, kiểm tra callback hoặc webhook trở về địa chỉ HTTPS công khai. Đây là các địa chỉ nhận kết quả hoặc thông báo từ dịch vụ bên ngoài.

Lưu ý: seeder không tạo đơn hàng, vì vậy có sản phẩm mẫu chưa đồng nghĩa với có doanh thu trong Finance. Đối với thanh toán và vận chuyển, cần dùng khóa cùng môi trường với địa chỉ API; API là giao diện để ứng dụng trao đổi dữ liệu với dịch vụ khác.

Các lệnh kiểm tra tại máy:

```
php artisan config:clear  
php artisan route:list  
php artisan test
```

Bộ kiểm thử hiện cấu hình SQLite trong bộ nhớ, vì vậy kết quả kiểm thử không thay cho việc kiểm tra kết nối Aiven thật. Có thể kiểm tra health check công khai từ PowerShell sau khi thay tên dịch vụ:

```
(Invoke-WebRequest -Uri "https://YOUR-SERVICE.onrender.com/up" -UseBasicParsing).StatusCode
```

Kết quả mong đợi là mã HTTP 200. Tiếp tục kiểm tra nghiệp vụ trên trình duyệt.

Trích cấu hình địa chỉ callback MoMo trên Render:

```
MOMO_REDIRECT_URL=https://YOUR-SERVICE.onrender.com/payment/momo/callback  
MOMO_IPN_URL=https://YOUR-SERVICE.onrender.com/payment/momo/ipn  
MOMO_VERIFY_SSL=true  
GHN_VERIFY_SSL=true
```

Cập nhật địa chỉ nhận thông báo GHN trên dịch vụ tương ứng thành đường dẫn /ghn/webhook của website HTTPS. Giữ các cơ chế kiểm tra thông báo trong ứng dụng; việc đường dẫn truy cập được chưa đủ để xác nhận một giao dịch hợp lệ.

Bước 9. Cập nhật sau khi đã triển khai

1. Sửa và kiểm tra tại local.
2. Xem lại thay đổi, tạo commit và đẩy lên GitHub.
3. Theo dõi triển khai tự động nếu đã bật; nếu chưa, chủ động triển khai phiên bản mới trên Render.
4. Đối chiếu commit Render đang dùng với commit vừa đưa lên.
5. Kiểm tra lại log và các chức năng chịu ảnh hưởng.

Lưu ý: sao lưu trước những thay đổi có ảnh hưởng dữ liệu. Không xóa toàn bộ bảng để khắc phục lỗi triển khai. Quay lại phiên bản mã nguồn cũ không tự khôi phục trạng thái cũ của cơ sở dữ liệu.

Ví dụ cập nhật đúng hai tập tin tài liệu của lần này — chạy tại máy khi bạn muốn đưa chúng lên GitHub:

```
git status --short
git add docs/HUONG_DAN_DEPLOY_LAR_VIDU2_CO_MA_NGUON.md
git add docs/HUONG_DAN_DEPLOY_LAR_VIDU2_CO_MA_NGUON.docx
git diff --cached --stat
git commit -m "Document Render deployment with code examples"
git push origin main
git log -1 --oneline
```

Nếu đang cập nhật mã ứng dụng, chọn các tập tin sửa tương ứng thay cho hai tài liệu trên. Lệnh cuối hiển thị commit mới nhất để đối chiếu với Render. Những lệnh này chỉ được minh họa trong tài liệu; việc cập nhật tài liệu không tự đẩy thay đổi lên GitHub.

--